UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Dynamic Network Slicing in architetture SDN

NCIS - Networks and Cloud Infrastuctures a.a. 2025-26

Prof. **Giorgio Ventre**
Prof. **Roberto Canonico**
Prof. **Alessio Botta**
Prof. **Giovanni Stanco**

Studentesse
Vittoria Alberto M63001750
Cristina Pecoraro DE9000143

Contents

Introduzione **1**

1 Topology Slicing e Design della Rete **2**

1.1 Topologia dell'Infrastruttura 2

1.2 Logica di Isolamento L2/L3 3

1.3 Validazione Sperimentale 4

2 Service Slicing e Slicing Dinamico **5**

2.1 Architettura e Gestione delle Priorità 5

2.2 Validazione Sperimentale 6

3 Dashboard di Monitoraggio e Classificazione **7**

3.1 Telemetria e Live Monitor 7

3.2 Classificazione Statistica dei Flussi (K-Means) 8

4 Conclusioni **9**

Introduzione

Il presente elaborato descrive lo sviluppo e la validazione di un'architettura SDN orientata al Network Slicing, implementata avvalendosi dell'emulatore Mininet e del controller Ryu tramite il protocollo OpenFlow 1.3. L'obiettivo principale è partizionare un'unica infrastruttura fisica in molteplici reti virtuali isolate, al fine di garantire requisiti stringenti di isolamento e Quality of Service (QoS) per flussi eterogenei.

L'architettura implementata si fonda su due concetti chiave:

- **Topology Slicing:** assicura un rigido isolamento logico di livello L2/L3 tra gruppi di host, confinandoli su percorsi di rete asimmetrici.
- **Service Slicing:** garantisce l'instradamento prioritario per il traffico critico (video) rispetto al traffico dati best-effort.

Al fine di superare i limiti dei tradizionali approcci statici, il sistema integra inoltre tre funzionalità ingegneristiche avanzate:

- **Slicing Dinamico:** un meccanismo di monitoraggio attivo del throughput che permette di riallocare reattivamente il traffico dati sui percorsi ad alta velocità durante i periodi di inattività della rete.
- **Classificazione tramite Machine Learning:** un modello non supervisionato (K-Means) che identifica i flussi analizzando esclusivamente le statistiche fisiche dei pacchetti, superando il limitante matching statico sulle porte di livello di trasporto.
- **Dashboard di Monitoraggio:** un'interfaccia grafica applicativa che fornisce la telemetria di rete e la visualizzazione della topologia dinamica in tempo reale.

Il documento è strutturato seguendo l'evoluzione logica del sistema. Nei prossimi capitoli verrà analizzato il design della topologia e la logica di isolamento. Successivamente, verranno illustrate le politiche di instradamento dinamico e, infine, si approfondirà l'integrazione del Machine Learning e della telemetria visiva.

Chapter 1

Topology Slicing e Design della Rete

1.1 Topologia dell'Infrastruttura

La rete emulata tramite Mininet è stata ingegnerizzata per supportare la coesistenza di flussi eterogenei con requisiti di banda differenti. La topologia si compone di 4 host (da H1 a H4) e 6 switch OpenFlow (da S1 a S6), orchestrati centralmente da un controller remoto Ryu. L'infrastruttura implementa due percorsi asimmetrici (slice) tra gli switch di bordo (S1 e S6) per supportare flussi eterogenei:

- **Upper Slice (Percorso Video):** Composto dagli switch S2 e S4, con link configurati a 10 Mbps per ospitare traffico multimediale ad alta densità.
- **Lower Slice (Percorso Dati):** Composto dagli switch S3 e S5, limitato a una capacità di 1 Mbps, destinato al traffico best-effort di background.

L'allocazione delle risorse di banda sui link fisici è stata implementata tramite le API `TCLink` di Mininet, come illustrato in Figura 1.1.

```
# Upper slice (Percorso video ad alta velocità: s1 -> s2 -> s4 -> s6)
self.net.addLink(self.s1, self.s2, bw=10)
self.net.addLink(self.s2, self.s4, bw=10)
self.net.addLink(self.s4, self.s6, bw=10)

# Lower slice (Percorso dati standard a bassa velocità: s1 -> s3 -> s5 -> s6)
self.net.addLink(self.s1, self.s3, bw=1)
self.net.addLink(self.s3, self.s5, bw=1)
self.net.addLink(self.s5, self.s6, bw=1)
```

Figure 1.1: Definizione dei link e della banda in Mininet.

1.2 Logica di Isolamento L2/L3

Il *Topology Slicing* richiede un rigoroso confinamento logico: H1 deve poter comunicare esclusivamente con H3, e H2 esclusivamente con H4.

Per soddisfare tale vincolo, il controller SDN applica una politica di forwarding deterministico basata sul filtraggio degli indirizzi MAC sorgente e destinazione. Dopo aver iniettato una regola proattiva di *table-miss*, che inoltra i pacchetti non riconosciuti al controller, il gestore degli eventi `Packet-In` stabilisce le porte di uscita in base all'identità degli host:

- Il traffico tra H1 e H3 viene forzato sulle interfacce collegate all'Upper Slice.
- Il traffico tra H2 e H4 viene vincolato alle interfacce della Lower Slice.

Il routing deterministico valuta in tempo reale l'indirizzo sorgente per deviare il traffico sui rami appropriati:

```
# --- UPPER SLICE (H1 <-> H3) ---
if src in ['00:00:00:00:00:01', '00:00:00:00:00:03']:
    if src == '00:00:00:00:00:01': # Da H1 verso H3
        if dpid == 1: out_port = 3
        elif dpid == 2: out_port = 2
        elif dpid == 4: out_port = 2
        elif dpid == 6: out_port = 1 # H3 si trova sulla porta 1 di S6
    elif src == '00:00:00:00:00:03': # Da H3 verso H1
        if dpid == 6: out_port = 3 # S4 si trova sulla porta 3 di S6
        elif dpid == 4: out_port = 1
        elif dpid == 2: out_port = 1
        elif dpid == 1: out_port = 1

# --- LOWER SLICE (H2 <-> H4) DINAMICO ---
elif src in ['00:00:00:00:00:02', '00:00:00:00:00:04']:
    if src == '00:00:00:00:00:02': # Da H2 verso H4
        if self.fast_lane_active:
            if dpid == 1: out_port = 3
            elif dpid == 2: out_port = 2
            elif dpid == 4: out_port = 2
            elif dpid == 6: out_port = 2 # H4 si trova sulla porta 2 di S6
        else:
            if dpid == 1: out_port = 4
            elif dpid == 3: out_port = 2
            elif dpid == 5: out_port = 2
            elif dpid == 6: out_port = 2 # H4 si trova sulla porta 2 di S6
    elif src == '00:00:00:00:00:04': # Da H4 verso H2
        if self.fast_lane_active:
            if dpid == 6: out_port = 3 # S4 (Fast Lane) è sulla porta 3 di S6
            elif dpid == 4: out_port = 1
            elif dpid == 2: out_port = 1
            elif dpid == 1: out_port = 2
        else:
            if dpid == 6: out_port = 4 # S5 (Slow Lane) è sulla porta 4 di S6
            elif dpid == 5: out_port = 1
            elif dpid == 3: out_port = 1
            elif dpid == 1: out_port = 2
```

Figure 1.2: Logica di inoltro e isolamento basata sui MAC address

I pacchetti diretti a destinazioni non autorizzate (es. da H1 verso H2 o H4) non soddisfano le condizioni di routing e vengono scartati (*drop*), garantendo la completa segregazione logica e fisica dei domini di broadcast.

1.3 Validazione Sperimentale

Il corretto instradamento e l'efficacia delle policy di isolamento sono state validate tramite due prove complementari.

In primo luogo, il test di raggiungibilità tramite il comando `pingall` (Figura 1.3) conferma il rigoroso partizionamento della rete: le comunicazioni incrociate tra slice differenti generano un *drop* totale, attestando la continuità esclusiva tra i nodi autorizzati.

```
*** Avvio CLI di Mininet
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
```

Figure 1.3: Risultato del pingall: i drop confermano il partizionamento logico della rete.

In secondo luogo, per dimostrare l'effettivo transito fisico dei pacchetti sui link designati, è stata effettuata una cattura del traffico tramite l'analizzatore `tshark` sull'interfaccia `s2-eth1` (nodo di ingresso dell'Upper Slice). Durante la generazione di traffico ICMP tra H1 e H3, il monitoraggio ha confermato che il traffico isolato viaggia esclusivamente sul percorso ad alta velocità designato.

```
vittoria@vittoria-VirtualBox:~/Documenti/NCIS$ sudo tshark -i s2-eth1 -f "icmp"
[sudo] password di vittoria:
Running as user "root" and group "root". This could be dangerous.
Capturing on 's2-eth1'
    1 0.000000000     10.0.0.1 → 10.0.0.3     ICMP 98 Echo (ping) request  id=0x15eb, seq=2/512, ttl=64
    2 0.000058500     10.0.0.3 → 10.0.0.1     ICMP 98 Echo (ping) reply    id=0x15eb, seq=2/512, ttl=64 (request in 1)
    3 1.023836490     10.0.0.1 → 10.0.0.3     ICMP 98 Echo (ping) request  id=0x15eb, seq=3/768, ttl=64
    4 1.023924822     10.0.0.3 → 10.0.0.1     ICMP 98 Echo (ping) reply    id=0x15eb, seq=3/768, ttl=64 (request in 3)
    5 2.025604115     10.0.0.1 → 10.0.0.3     ICMP 98 Echo (ping) request  id=0x15eb, seq=4/1024, ttl=64
    6 2.025705127     10.0.0.3 → 10.0.0.1     ICMP 98 Echo (ping) reply    id=0x15eb, seq=4/1024, ttl=64 (request in 5)
    7 3.036753500     10.0.0.1 → 10.0.0.3     ICMP 98 Echo (ping) request  id=0x15eb, seq=5/1280, ttl=64
    8 3.036833724     10.0.0.3 → 10.0.0.1     ICMP 98 Echo (ping) reply    id=0x15eb, seq=5/1280, ttl=64 (request in 7)
```

Figure 1.4: Cattura tramite `tshark` sull'interfaccia `s2-eth1`.

Chapter 2

Service Slicing e Slicing Dinamico

2.1 Architettura e Gestione delle Priorità

Oltre all'isolamento topologico, la rete è stata progettata per differenziare il trattamento dei flussi in base alla tipologia di servizio (*Service Slicing*). L'obiettivo principale è garantire che il traffico multimediale (Video), caratterizzato da stringenti requisiti di banda e latenza, goda costantemente di priorità assoluta sull'infrastruttura ad alte prestazioni, per cui: il traffico multimediale (Video) gode costantemente di priorità sull'*Upper Slice* (10 Mbps), mentre il traffico dati *best-effort* è destinato nativamente alla *Lower Slice* (1 Mbps).

Tuttavia, confinare staticamente il traffico dati sul percorso più lento porta inevitabilmente a un sottoutilizzo delle risorse fisiche quando il flusso video è assente. Per ovviare a questo limite, è stato ingegnerizzato un meccanismo di *Slicing Dinamico*. Il controller SDN è stato programmato per operare come un sistema di telemetria a circuito chiuso, capace di valutare il carico di rete al volo: tramite un thread dedicato invia periodicamente messaggi `OFPFlowStatsRequest` allo switch di ingresso S1, estrae il `byte_count` associato al traffico video e ne calcola il throughput effettivo in Mbps. Sulla base di questo valore, il controller adatta dinamicamente le politiche di routing dinamico.

Come evidenziato nel codice (Figura 2.1), le regole di commutazione si basano su una soglia prefissata:

- **Video Inattivo (< 0.5 Mbps):** Se il traffico video è assente o sotto soglia, il controller abilita la cosiddetta *Fast Lane*. Invia istruzioni OpenFlow di tipo `OFPFC_DELETE` per rimuovere le vecchie rotte. Questo forza gli switch a richiedere nuove *flow entry* che devieranno i flussi di H2-H4 sull'Upper Slice, sfruttando la banda libera a 10 Mbps.

- **Video Attivo ($\geq$ 0.5 Mbps):** Al superamento della soglia, il controller reagisce istantaneamente eliminando l'accesso alla Fast Lane per il traffico dati, instradandolo nuovamente sulla Lower

Slice a 1 Mbps e tutelando il flusso video.

```python
# LOGICA DI SLICING DINAMICO SU S1
if dpid == 1:
    current_video_bytes = 0
    for stat in body:
        if stat.match.get('eth_src') == '00:00:00:00:00:01' and stat.match.get('eth_dst') == '00:00:00:00:00:03':
            current_video_bytes = stat.byte_count

    # Calcolo banda video (Bytes passati negli ultimi 10 sec convertiti in Mbps)
    diff = current_video_bytes - self.video_bytes_prev
    self.video_bytes_prev = current_video_bytes
    bw_mbps = (diff * 8) / (10 * 1000000)

    # Se Video < 0.5 Mbps, abilita la Fast Lane per i Dati
    if bw_mbps < 0.5 and not self.fast_lane_active:
        self.logger.info("SLICING DINAMICO: Video Inattivo (%.2f Mbps). Sposto Dati su Slice 10Mbps", bw_mbps)
        self.fast_lane_active = True
        self._reroute_data_slice()

    # Se Video >= 0.5 Mbps, riduci i Dati sulla Slow Lane
    elif bw_mbps >= 0.5 and self.fast_lane_active:
        self.logger.info("SLICING DINAMICO: Video Attivo (%.2f Mbps). Ritorno Dati su Slice 1Mbps", bw_mbps)
        self.fast_lane_active = False
        self._reroute_data_slice()
```

Figure 2.1: Calcolo della banda e attivazione della Fast Lane.

2.2 Validazione Sperimentale

L'efficacia dell'orchestrazione dinamica è stata validata generando traffico concorrente.

L'analisi tramite `tshark` sull'interfaccia `s3-eth1` (Lower Slice, Figura 2.2) dimostra il corretto spostamento dei flussi. Quando la Fast Lane è attiva, l'interfaccia non registra pacchetti dati. Non appena viene iniettato il traffico video, il controller devia reattivamente i flussi TCP su questo percorso, i quali appaiono immediatamente nel terminale di cattura. L'intera transizione è certificata dai log del controller (Figura 2.3), che documentano i calcoli di banda e l'aggiornamento delle regole OpenFlow in tempo reale.

```
1660 19.561165594    10.0.0.4 → 10.0.0.2    TCP 66 5001 → 41426 [ACK] Seq=1 Ack=2322593 Win=502 Len=0 TSval=3669010216 TSecr=3845614237
1661 19.584913277    10.0.0.2 → 10.0.0.4    TCP 2962 41426 → 5001 [PSH, ACK] Seq=2322593 Ack=1 Win=83 Len=2896 TSval=3845614261 TSecr=3669010167
1662 19.586123713    10.0.0.4 → 10.0.0.2    TCP 66 5001 → 41426 [ACK] Seq=1 Ack=2325489 Win=502 Len=0 TSval=3669010241 TSecr=3845614261
1663 19.608994069    10.0.0.2 → 10.0.0.4    TCP 2962 41426 → 5001 [PSH, ACK] Seq=2325489 Ack=1 Win=83 Len=2896 TSval=3845614286 TSecr=3669010216
1664 19.609866176    10.0.0.4 → 10.0.0.2    TCP 66 5001 → 41426 [ACK] Seq=1 Ack=2328385 Win=502 Len=0 TSval=3669010264 TSecr=3845614286
1665 19.633328342    10.0.0.2 → 10.0.0.4    TCP 2962 41426 → 5001 [PSH, ACK] Seq=2328385 Ack=1 Win=83 Len=2896 TSval=3845614311 TSecr=3669010241
1666 19.633764753    10.0.0.4 → 10.0.0.2    TCP 66 5001 → 41426 [ACK] Seq=1 Ack=2331281 Win=502 Len=0 TSval=3669010288 TSecr=3845614311
1667 19.659663652    10.0.0.2 → 10.0.0.4    TCP 2962 41426 → 5001 [PSH, ACK] Seq=2331281 Ack=1 Win=83 Len=2896 TSval=3845614334 TSecr=3669010241
```

Figure 2.2: Cattura del traffico tramite `tshark` su `s3-eth1`.

```
vittoria@vittoria-VirtualBox:~/Documenti/NCIS$ ryu-manager main_controller.py
loading app main_controller.py
loading app ryu.controller.ofp_handler
instantiating app main_controller.py of SlicingMonitorController
instantiating app ryu.controller.ofp_handler of OFPHandler
SLICING DINAMICO: Video Inattivo (0.00 Mbps). Sposto Dati su Slice 10Mbps!
SLICING DINAMICO: Video Attivo (4.31 Mbps). Ritorno Dati su Slice 1Mbps!
```

Figure 2.3: Log del controller SDN: rilevazione della banda e riscrittura dinamica delle rotte.

Chapter 3

Dashboard di Monitoraggio e Classificazione

Per soddisfare i requisiti avanzati di osservabilità e analisi del traffico, l'architettura è stata estesa con un'interfaccia grafica, sviluppata tramite il framework Python *Streamlit*. L'applicativo opera in totale disaccoppiamento dal controller Ryu, leggendo in concorrenza il dataset telemetrico per non penalizzare le performance di instradamento del Control Plane. Le funzionalità della dashboard si articolano in due moduli chiave.

3.1 Telemetria e Live Monitor

Il primo modulo elabora il flusso continuo di dati OpenFlow applicando una finestra temporale mobile (sliding window) basata sugli ultimi campioni registrati. L'interfaccia (Figura 3.1) offre un riscontro visivo immediato delle decisioni di routing prese dal piano di controllo. Nello specifico, il sistema traccia:

- **Throughput in tempo reale (Mbps):** permette di monitorare simultaneamente le performance del traffico video (Upper Slice) e dei dati best-effort (Lower Slice) tramite grafici ad area.
- **Stato del Dynamic Slicing:** un indicatore metrico segnala reattivamente l'attivazione della *Fast Lane* e l'attuale soglia di commutazione in uso.

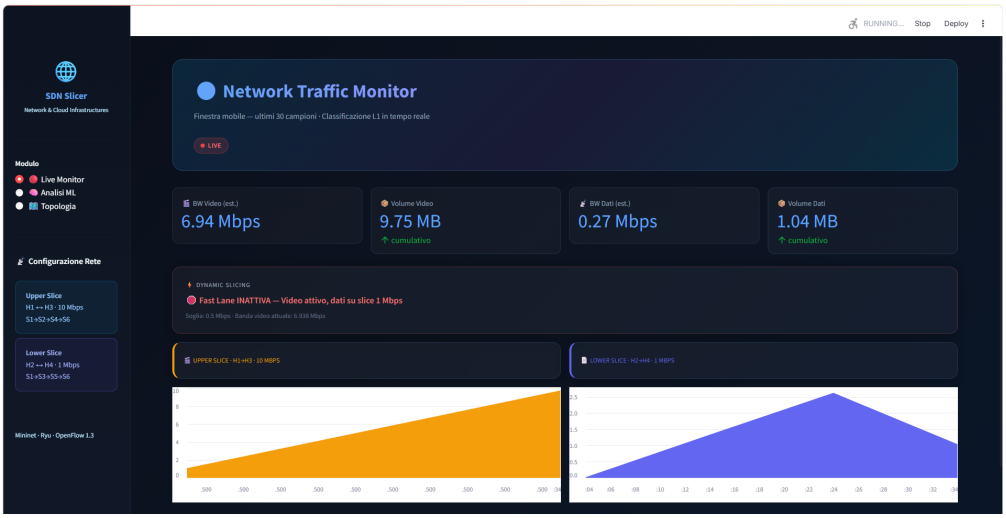


Figure 3.1: Interfaccia di Live Monitor: visualizzazione in tempo reale del throughput e dei volumi cumulativi sulle rispettive slice.

3.2 Classificazione Statistica dei Flussi (K-Means)

Il secondo modulo integra l'elaborazione del traffico tramite Machine Learning non supervisionato. Per superare le limitazioni di un approccio statico basato unicamente sulla lettura delle porte di livello di trasporto (es. porta UDP/TCP), il sistema estrae proattivamente le metriche fisiche dal Data Plane, in particolare il volume di byte (`byte_count`) e il conteggio dei pacchetti (`packet_count`).

Questi dati vengono elaborati da un algoritmo K-Means, il quale raggruppa i flussi in modo autonomo analizzandone esclusivamente le densità e i pattern statistici. L'adozione di questo modello ha dimostrato un'elevata coerenza nel discriminare con precisione i flussi ad alta priorità (video) da quelli best-effort (dati) senza alcuna ispezione delle intestazioni L4, confermando l'efficacia di un'architettura di rete pienamente Cloud-Native.

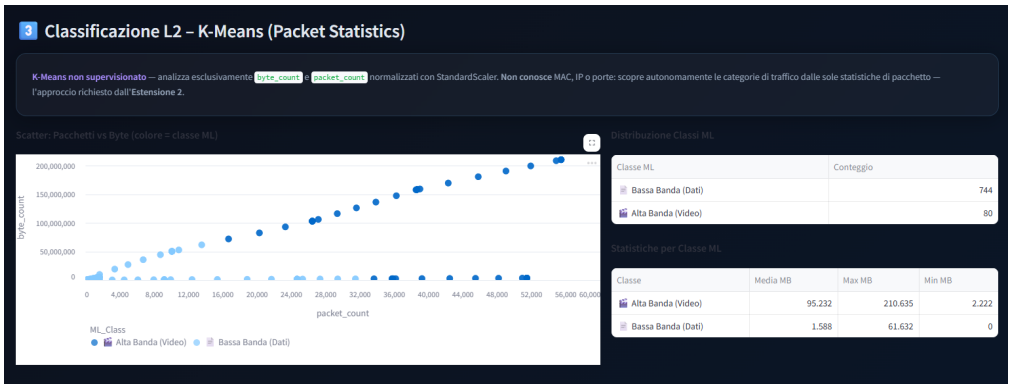


Figure 3.2: Interfaccia di Analisi ML: Classificazione tramite K-Means.

Chapter 4

Conclusioni

L'obiettivo di questo progetto era la progettazione e l'implementazione di un'architettura SDN in grado di gestire e segregare dinamicamente il traffico in un ambiente virtualizzato. I risultati sperimentali confermano il pieno raggiungimento dei requisiti di base e il superamento delle aspettative tramite l'integrazione sinergica delle tre estensioni avanzate richieste.

L'infrastruttura ha dimostrato un rigoroso confinamento L2/L3 dei flussi (*Topology Slicing*), garantendo al contempo l'orchestrazione corretta delle priorità per il traffico multimediale (*Service Slicing*). Il vero valore aggiunto del sistema risiede tuttavia nella sua reattività: il meccanismo di *Dynamic Slicing* ha permesso alla rete di auto-ottimizzarsi al volo, riallocando il traffico dati per massimizzare l'efficienza dei link fisici senza mai compromettere la Quality of Service dei flussi critici.

Infine, l'adozione dell'algoritmo di Machine Learning K-Means per la classificazione puramente statistica dei pacchetti, unita allo sviluppo di una telemetria visiva e interattiva in tempo reale tramite la Dashboard, ha trasformato l'elaborato da un classico task di networking a una moderna architettura allineata ai più recenti paradigmi Cloud-Native.